

TUGAS PRAKTIKUM 5 “SINKRONISASI SISTEM OPRASI

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:

Nama :Azqi Firman Maulana

NIM : 24106510564

Kelas : Kamis-C

PROGRAM 1 — DEADLOCK VERSION

(dining_deadlock.c)

Langkah Langkah: Buatlah sebuah file untuk isi program bernama nano dining_deadlock.c dengan isi kode ada pada gambar dibawah ini, lalu compile dan jalankan program sehingga hasilnya nampak seperti berikut:

```
azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ nano deadlock_solution.c
azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ nano dining_philosophers_deadlock.c
azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ gcc -pthread -o deadlock dinin
g_philosophers_deadlock.c
./deadlock
🤔 Filsuf-1 sedang berpikir...
🤔 Filsuf-2 sedang berpikir...
🤔 Filsuf-3 sedang berpikir...
🤔 Filsuf-4 sedang berpikir...
🤔 Filsuf-5 sedang berpikir...
🍴 Filsuf-1 lapar, mencoba ambil sumpit...
🍴 Filsuf-2 lapar, mencoba ambil sumpit...
// Filsuf-2 mengambil sumpit kiri (1)
🍴 Filsuf-3 lapar, mencoba ambil sumpit...
// Filsuf-3 mengambil sumpit kiri (2)
// Filsuf-1 mengambil sumpit kiri (0)
🍴 Filsuf-4 lapar, mencoba ambil sumpit...
// Filsuf-4 mengambil sumpit kiri (3)
🍴 Filsuf-5 lapar, mencoba ambil sumpit...
// Filsuf-5 mengambil sumpit kiri (4)
^C
```

```

azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ nano dining_philosophers_ordering.c
azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ gcc -pthread -o ordering dinin
g_philosophers_ordering.c
./ordering
🤖 Filsuf-1 berpikir...
🤖 Filsuf-2 berpikir...
🤖 Filsuf-3 berpikir...
🤖 Filsuf-4 berpikir...
🤖 Filsuf-5 berpikir...
🍽️ Filsuf-1 lapar...
🍽️ Filsuf-1 MAKAN...
🍽️ Filsuf-2 lapar...
🍽️ Filsuf-3 lapar...
🍽️ Filsuf-4 lapar...
🍽️ Filsuf-4 MAKAN...
🍽️ Filsuf-5 lapar...
🤖 Filsuf-1 berpikir...
🍽️ Filsuf-2 MAKAN...
🤖 Filsuf-4 berpikir...
🍽️ Filsuf-5 MAKAN...
🍽️ Filsuf-1 lapar...
🤖 Filsuf-2 berpikir...
🍽️ Filsuf-3 MAKAN...
🍽️ Filsuf-4 lapar...
🤖 Filsuf-5 berpikir...
🍽️ Filsuf-1 MAKAN...
🍽️ Filsuf-2 lapar...
🤖 Filsuf-1 berpikir...
🤖 Filsuf-3 berpikir...
🍽️ Filsuf-2 MAKAN...
🍽️ Filsuf-5 lapar...
🍽️ Filsuf-4 MAKAN...
🍽️ Filsuf-3 lapar...
🍽️ Filsuf-1 lapar...
🍽️ Filsuf-1 MAKAN...
🤖 Filsuf-2 berpikir...
🤖 Filsuf-4 berpikir...
🍽️ Filsuf-3 MAKAN...
🍽️ Filsuf-4 lapar...
🍽️ Filsuf-2 lapar...
🤖 Filsuf-1 berpikir...

```

```

azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ nano dining_philosophers_limit
c
azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ gcc -pthread -o limit dinin
g_philosophers_limit.c -lrt
./limit
🤖 Filsuf-1 berpikir...
🤖 Filsuf-2 berpikir...
🤖 Filsuf-3 berpikir...
🤖 Filsuf-4 berpikir...
🤖 Filsuf-5 berpikir...
🍽️ Filsuf-1 lapar...
🍽️ Filsuf-1 MAKAN...
🍽️ Filsuf-2 lapar...
🍽️ Filsuf-3 lapar...
🍽️ Filsuf-3 MAKAN...
🍽️ Filsuf-4 lapar...
🍽️ Filsuf-5 lapar...
🤖 Filsuf-1 berpikir...
🍽️ Filsuf-5 MAKAN...
🍽️ Filsuf-2 MAKAN...
🤖 Filsuf-3 berpikir...
🍽️ Filsuf-1 lapar...
🍽️ Filsuf-3 lapar...
🤖 Filsuf-5 berpikir...
🤖 Filsuf-2 berpikir...
🍽️ Filsuf-1 MAKAN...
🍽️ Filsuf-4 MAKAN...
🍽️ Filsuf-5 lapar...
🍽️ Filsuf-2 lapar...
🤖 Filsuf-1 berpikir...
🍽️ Filsuf-5 MAKAN...
🍽️ Filsuf-3 MAKAN...
🤖 Filsuf-4 berpikir...
X 🍽️ Filsuf-4 lapar...
🍽️ Filsuf-1 lapar...
🤖 Filsuf-5 berpikir...
🍽️ Filsuf-4 MAKAN...
🤖 Filsuf-3 berpikir...
🍽️ Filsuf-2 MAKAN...
7

```

Bisa kita lihat berdasarkan gambar diatas bahwa setiap filsuf mengambil sumpit kiri lalu menunggu kanan sehingga terbentuk siklus saling menunggu (1 menunggu 2, 2 menunggu 3, ..., 5 menunggu 1) dan tidak ada yang dapat melanjutkan yang mengakibatkan program menjadi hang.

PROGRAM 2 RESOURCE ORDERING (dining_resource_ordering.c) (Solusi 1 untuk deadlock)

Langkah Langkah: Buatlah sebuah file untuk isi program bernama dining_resource_ordering.c dengan isi kodenya seperti gambar dibawah, lalu compile dan jalankan selama 60 detik sehingga hasilnya seperti pada gambar dibawah gambar kode berikut ini:

```
// dining_resource_ordering.c
// Solusi 1: Resource Ordering (mencegah deadlock)
// Filsuf genap ambil sumpit kiri dulu, filsuf ganjil ambil sumpit kanan dulu

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

#define N 5
pthread_mutex_t chopstick[N];
int eat_count[N] = {0};
time_t start_time;

void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    while (time(NULL) - start_time < 60) {
        printf("😴 Filsuf-%d sedang berpikir...\n", id + 1);
        sleep(rand() % 2 + 1);

        printf("🍴 Filsuf-%d lapar...\n", id + 1);

        // 🔑 KUNCI SOLUSI: Genap ambil kiri dulu, ganjil ambil kanan dulu
        if (id % 2 == 0) {
            pthread_mutex_lock(&chopstick[left]);
            pthread_mutex_lock(&chopstick[right]);
        } else {
            pthread_mutex_lock(&chopstick[right]);
            pthread_mutex_lock(&chopstick[left]);
        }

        // MAKAN
        printf("🍽️ Filsuf-%d MAKAN...\n", id + 1);
        eat_count[id]++;
        sleep(2);
    }
}
```

```

        // Kembalikan sumpit
        pthread_mutex_unlock(&chopstick[left]);
        pthread_mutex_unlock(&chopstick[right]);
    }

    return NULL;
}

int main() {
    pthread_t tid[N];
    int id[N];

    srand(time(NULL));
    start_time = time(NULL);

    // Init mutex
    for (int i = 0; i < N; i++)
        pthread_mutex_init(&chopstick[i], NULL);

    // Buat thread filsuf
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &id[i]);
    }

    // Tunggu semua selesai
    for (int i = 0; i < N; i++)
        pthread_join(tid[i], NULL);

    // Statistik
    printf("\n=== STATISTIK (60 detik) ===\n");
    for (int i = 0; i < N; i++)
        printf("Filsuf-%d makan %d kali\n", i + 1, eat_count[i]);

    printf("✅ Solusi Resource Ordering berhasil!\n");
    printf("✅ Tidak ada deadlock!\n");

    return 0;
}

```

Output:

```

// deadlock
🤖 Filsuf-1 sedang berpikir...
🤖 Filsuf-2 sedang berpikir...
🤖 Filsuf-3 sedang berpikir...
🤖 Filsuf-4 sedang berpikir...
🤖 Filsuf-5 sedang berpikir...
🍷 Filsuf-1 lapar, mencoba ambil sumpit...
🍷 Filsuf-2 lapar, mencoba ambil sumpit...
// Filsuf-2 mengambil sumpit kiri (1)
🍷 Filsuf-3 lapar, mencoba ambil sumpit...
// Filsuf-3 mengambil sumpit kiri (2)
// Filsuf-1 mengambil sumpit kiri (0)
🍷 Filsuf-4 lapar, mencoba ambil sumpit...
// Filsuf-4 mengambil sumpit kiri (3)
🍷 Filsuf-5 lapar, mencoba ambil sumpit...
// Filsuf-5 mengambil sumpit kiri (4)

```

Bisa kita lihat berdasarkan gambar diatas yang menggunakan solusi Resource Ordering, setiap filsuf mengambil sumpit dalam urutan yang sama berdasarkan nomor terkecil terlebih dahulu. Dengan cara ini, tidak akan terbentuk siklus saling tunggu sehingga deadlock dapat dihindari sepenuhnya. Program berjalan stabil dan semua filsuf dapat makan bergantian tanpa macet.

PROGRAM 3 — LIMITING PHILOSOPHERS (dining_limited.c)

SEMAPHORE (Mengatur Akses Terlambat)

Langkah Langkah: Buatlah sebuah file untuk isi program bernama dining_limited.c dengan isi kodenya seperti gambar dibawah, lalu compile dan jalankan selama 60 detik sehingga hasilnya seperti pada gambar dibawah gambar kode berikut ini:

```
// dining_limited.c
// Solusi 2: Limiting Philosophers (mencegah deadlock)
// Menggunakan semaphore agar max 4 filsuf boleh mengambil sumpit

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>
#include <stdlib.h>
#include <time.h>

#define N 5

pthread_mutex_t chopstick[N];
sem_t limit; // Batas maksimal filsuf yang boleh bersamaan
int eat_count[N] = {0};
time_t start_time;

// Thread fungsi filsuf
void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    while (time(NULL) - start_time < 60) {
        printf("🤖 Filsuf-%d berpikir...\n", id + 1);
        sleep(rand() % 2 + 1);

        printf("🍲 Filsuf-%d lapar...\n", id + 1);

        // 🔑 BATASAN SEMAPHORE: hanya 4 filsuf boleh "masuk ke area makan"
        sem_wait(&limit);

        // Ambil kedua sumpit
        pthread_mutex_lock(&chopstick[left]);
        pthread_mutex_lock(&chopstick[right]);

        // Sedang makan
        printf("🍽️ Filsuf-%d MAKAN...\n", id + 1);
        eat_count[id]++;
        sleep(1);
    }
}
```

```

        eat_count[id]++;
        sleep(2);

        // Kembalikan sumpit
        pthread_mutex_unlock(&chopstick[left]);
        pthread_mutex_unlock(&chopstick[right]);

        // Keluarkan izin semaphore
        sem_post(&limit);
    }

    return NULL;
}

int main() {
    pthread_t tid[N];
    int id[N];

    srand(time(NULL));
    start_time = time(NULL);

    // Semaphore: hanya 4 filsuf boleh makan/menunggu makan
    sem_init(&limit, 0, 4);

    // Init chopstick (mutex)
    for (int i = 0; i < N; i++)
        pthread_mutex_init(&chopstick[i], NULL);

    // Buat thread filsuf
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &id[i]);
    }

    // Join semua thread
    for (int i = 0; i < N; i++)
        pthread_join(tid[i], NULL);

    // Tampilkan statistik hasil

```

```

    printf("\n=== STATISTIK (60 detik) - LIMITING PHILOSOPHERS ===\n");
    for (int i = 0; i < N; i++)
        printf("Filsuf-%d makan %d kali\n", i + 1, eat_count[i]);

    printf("✅ Tidak ada deadlock!\n");
    printf("✅ Solusi Limiting Philosophers berhasil!\n");

    return 0;
}

```

Output:

```
azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ nano dining_limited.c
azqi@MSI:~/praktikum-sinkronisasi/percobaan3$ gcc -pthread -o dining_limited
dining_limited.c -lrt
./dining_limited
🤖 Filsuf-1 berpikir...
🤖 Filsuf-2 berpikir...
🤖 Filsuf-3 berpikir...
🤖 Filsuf-4 berpikir...
🤖 Filsuf-5 berpikir...
🍽️ Filsuf-1 lapar...
🍽️ Filsuf-1 MAKAN...
🍽️ Filsuf-2 lapar...
🍽️ Filsuf-3 lapar...
🍽️ Filsuf-3 MAKAN...
🍽️ Filsuf-4 lapar...
🍽️ Filsuf-5 lapar...
🤖 Filsuf-1 berpikir...
🍽️ Filsuf-5 MAKAN...
🤖 Filsuf-3 berpikir...
🍽️ Filsuf-2 MAKAN...
🍽️ Filsuf-1 lapar...
🍽️ Filsuf-3 lapar...
🍽️ Filsuf-4 MAKAN...
🤖 Filsuf-5 berpikir...
🤖 Filsuf-2 berpikir...
🍽️ Filsuf-1 MAKAN...
🍽️ Filsuf-5 lapar...
🤖 Filsuf-4 berpikir...
🍽️ Filsuf-3 MAKAN...
🍽️ Filsuf-2 lapar...
🤖 Filsuf-1 berpikir...
🍽️ Filsuf-5 MAKAN...
🍽️ Filsuf-1 lapar...
🍽️ Filsuf-4 lapar...
🤖 Filsuf-3 berpikir...
🍽️ Filsuf-4 MAKAN...
🤖 Filsuf-5 berpikir...
🍽️ Filsuf-2 MAKAN...
🍽️ Filsuf-5 lapar...
🍽️ Filsuf-3 lapar...
🍽️ Filsuf-3 MAKAN...
🤖 Filsuf-2 berpikir...
```

Bisa kita lihat berdasarkan gambar diatas yang menggunakan Solusi Limiting Philosophers ia menggunakan semaphore untuk membatasi jumlah filsuf yang boleh mencoba mengambil sumpit secara bersamaan. Nilai semaphore diinisialisasi sebanyak N-1, artinya hanya empat filsuf (dari lima) yang diizinkan berada dalam kondisi “lapar” sekaligus. Dengan pembatasan ini, satu filsuf selalu menunggu di luar, sehingga tidak mungkin terbentuk siklus saling menunggu yang menyebabkan deadlock. Hasil pengujian menunjukkan bahwa program berjalan terus tanpa hang, dan semua filsuf dapat makan bergantian dengan adil.

TABEL EFISENSI/FAIRNESS | Solusi \ Filsuf | F1 | F2 | F3 | F4 | F5
| Total | | ResourceOrdering | 56 | 60 | 62 | 65 | 63 | 306 | | Limiting
| 58 | 63 | 55 | 64 | 58 | 298 | Berdasarkan tabel diatas, dari kedua

pendekatan yang diuji, solusi “Limiting Philosophers” merupakan metode yang paling adil (fair) karena semua filsuf mendapatkan kesempatan untuk makan secara bergantian tanpa mengalami kelaparan (starvation). sementara pada solusi Resource Ordering, walaupun deadlock dapat dicegah, terdapat kemungkinan beberapa filsuf terus kalah cepat mengambil sumpit dan makan lebih sedikit